

PROJECT SYNOPSIS

Safe Charging Chip for Mobile Phones

An ESP32 + FPGA (PYNQ-Z2) adapter safety controller that detects abnormal charging conditions and cuts power before a battery fire can start

Prisha

B.Tech, Electronics and Computer Engineering (Third Year)

October 2026

Contents

Contents	2
1. Abstract	3
2. Problem Statement and Motivation	3
3. Objectives	3
4. System Overview	3
5. Hardware Components	4
6. ESP32 Firmware (Arduino IDE)	5
7. FPGA Design (PYNQ-Z2, Verilog / Vivado)	6
8. Two-Layer Protection Concept	7
9. Testing Methodology and Test Plan	8
10. Limitations and Future Scope	8
11. Conclusion	9
Appendix A. Parameter Summary	9

1. Abstract

Mobile phone battery fires usually start with an ordinary charging session going wrong: an over-voltage from a faulty adapter, excess current, or a battery that keeps heating while it is still being charged. Most phones and cheap adapters rely on a single layer of protection, or none at all, in the charging path.

This project presents the **Safe Charging Chip**, a prototype that sits between a charging adapter and the phone battery. An **ESP32** continuously measures voltage, current and temperature, runs a latching safety state machine and drives a relay that disconnects the battery on a fault. In parallel, a **PYNQ-Z2 FPGA** receives the same measurements over UART and independently evaluates them with its own hardware safety FSM, adding warning detection for sudden voltage spikes, current surges and rapid temperature rise. Two independent decision-makers, one in software and one in hardware, make the system harder to defeat with a single failure.

The prototype is tested with a dummy battery, and fault conditions are injected with potentiometers and a manual test button so that every protection path can be exercised safely on the bench.

2. Problem Statement and Motivation

A lithium-ion battery that is overcharged, over-heated or charged with excess current can enter thermal runaway, where internal heating feeds itself and can end in venting, fire or explosion. Existing protection is typically a single protection IC inside the phone, and the adapter itself usually has no temperature awareness at all. Counterfeit or damaged adapters, hot environments and charging under pillows make this worse.

Problem: there is no inexpensive, independent, adapter-side safeguard that watches voltage, current and temperature together, reacts to the *trend* of those values and not only to fixed limits, and physically disconnects the battery when something is wrong.

3. Objectives

- Continuously monitor charging voltage, charging current and battery temperature.
- Cut off charging with a relay when any value crosses its safe limit or a sensor fails.
- Warn the user early (yellow LED, LCD) before a hard fault is reached.
- Detect abnormal *rates of change*: voltage spikes, current surges and rapid temperature rise.
- Provide two independent protection layers: ESP32 firmware and FPGA hardware logic.
- Make every protection path testable on the bench without risking a real battery.

4. System Overview

The ESP32 is the front end. It reads the three sensors every 500 ms, evaluates its limits, latches any fault, switches the relay, and updates the LCD, LEDs and buzzer. After every loop it also sends a one-line text frame with the latest measurements and its own status flags to the PYNQ-Z2. The FPGA decodes the frame in hardware, applies its own thresholds and anomaly checks, and drives its own LED, warning, fault and relay_cutoff outputs.

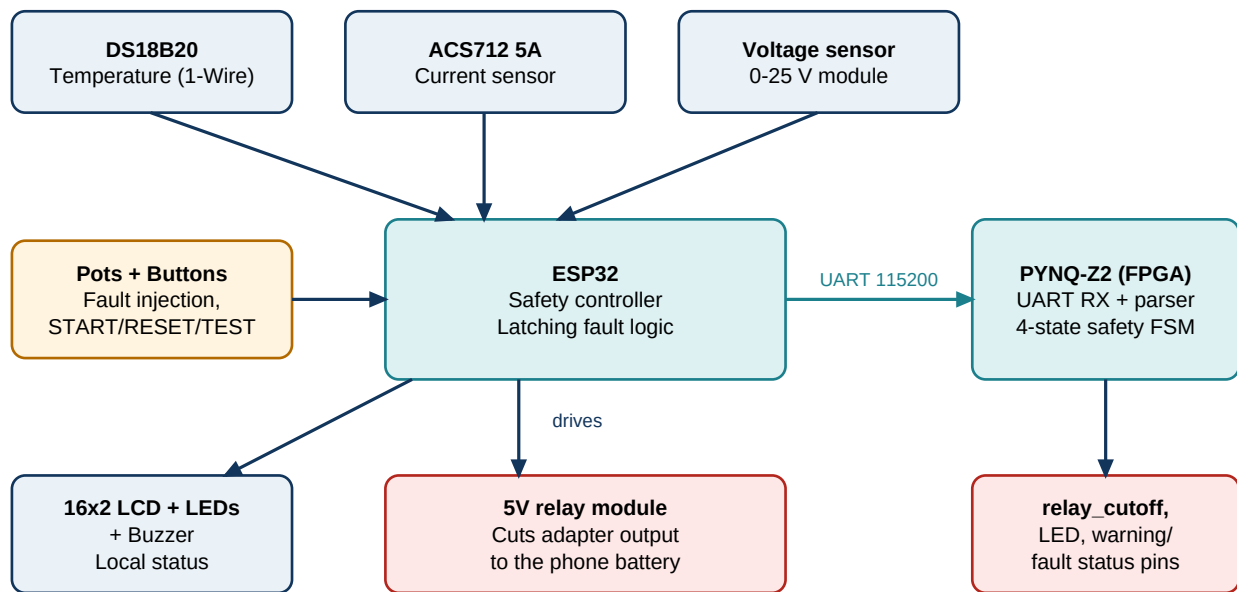


Figure 1. System block diagram

4.1 Operating principle

- **Sense:** DS18B20 (temperature), ACS712 (current) and a 0-25 V voltage module are sampled by the ESP32.
- **Decide (layer 1):** ESP32 compares against limits, latches a fault and opens the relay.
- **Decide (layer 2):** the FPGA independently evaluates the same data and raises warning or fault states.
- **Act:** relay disconnects the battery; LEDs, buzzer and LCD tell the user what happened.
- **Recover:** the ESP32 fault stays latched until the RESET button is pressed; the FPGA clears when readings are safe again.

5. Hardware Components

Component	Qty	Role in the project
PYNQ-Z2 board	1	FPGA platform running the hardware safety FSM (125 MHz clock)
ESP32 dev board	1	Sensor acquisition, latching fault logic, relay/LCD/LED/buzzer control, UART to FPGA
DS18B20 temperature sensor	1	Digital 1-Wire temperature measurement; disconnect is detected as a sensor fault
ACS712 5A current sensor module	1	Measures charging current (185 mV/A), output scaled to the ESP32 ADC range
0-25 V voltage sensor module	1	Measures charging voltage through a 5:1 divider
5V 1-channel relay module	1	Active-LOW relay that disconnects charging on a fault
16x2 I2C LCD (address 0x27)	1	Local display of readings and fault messages
LEDs (green, yellow, red)	3+	Normal / warning / fault indication
Buzzer	1	Audible alarm on a latched fault
Push buttons	4+	START, RESET, TEST (manual fault) and MENU
10k potentiometers	3	Fault injection for voltage, current and temperature during testing
3-channel level/voltage converter	3	Interface between 5 V modules and 3.3 V logic

Component	Qty	Role in the project
Dummy battery and adapter	1+1	Safe test load and power source
Diodes 1N4007	2	Protection in the power path
Capacitors 100 nF / 10 uF	5 / 2	Decoupling and noise filtering
Resistors 220 ohm / 10k	10 / 5	LED current limiting and pull-ups / dividers
2 x 1.5 V batteries	2	Additional working supply for the prototype

6. ESP32 Firmware (Arduino IDE)

6.1 Pin assignment

Function	GPIO	Notes
DS18B20 data	19	1-Wire, 10-bit resolution (0.25 deg C steps)
ACS712 output	35	ADC, 11 dB attenuation
Voltage sensor	32	ADC, 11 dB attenuation
Fault pots: voltage / current / temperature	33 / 36 / 39	Read as 0-100 %
Relay	26	Active LOW (LOW = relay ON, HIGH = cut off)
Buzzer	25	HIGH = alarm
LEDs green / yellow / red	2 / 4 / 15	Status
Buttons START / RESET / TEST / MENU	27 / 14 / 13 / 23	INPUT_PULLUP, active LOW
LCD I2C (SDA / SCL)	21 / 22	Address 0x27, 16x2
UART2 to PYNQ (RX2 / TX2)	16 / 17	115200 baud, 8N1

6.2 Measurement and calibration

- **Voltage:** ADC reading in millivolts (`analogReadMilliVolts`) multiplied by 5.0 for the 0-25 V module (5:1 divider).
- **Current:** $I = [(V_{\text{adc}} - 1749.5 \text{ mV}) \times 1.5 / 185]$. The 1749.5 mV is the measured zero-current level, 185 mV/A is the ACS712-5A sensitivity and 1.5 corrects the external divider that brings the sensor output into the ESP32 range. The absolute value is used because current direction does not matter for this prototype.
- **Temperature:** DS18B20 read with `requestTemperatures()`; a disconnected sensor returns -127 deg C and sets `tempSensorFault`.
- **Power:** $P = V \times I$, shown on the LCD.
- **Sampling:** sensors and safety checks every 500 ms; LCD refreshed every 2 s.

6.3 Safety logic

The ESP32 treats a condition as a **fault** if any of the following is true: voltage above 5.5 V, current above 7.5 A, temperature above 40.0 deg C, DS18B20 disconnected, any fault pot at or above 90 %, or the manual TEST button pressed. A fault sets `faultLatched`, which stays set even if the reading returns to normal, until the RESET button is pressed. A **warning** is raised when temperature is at or above 32.0 deg C or any pot is at or above 70 %.

State	Relay	Green	Yellow	Red	Buzzer	LCD
Not started	OFF	off	off	off	off	SYSTEM READY / Press START
Normal	ON	on	off	off	off	V, I, T, P readings

State	Relay	Green	Yellow	Red	Buzzer	LCD
Warning	ON	off	on	off	off	V, I, T, P readings
Fault (latched)	OFF	off	off	on	on	!! FAULT !! / RELAY CUT OFF
Temp sensor error	OFF	off	off	on	on	TEMP SENSOR ERR

6.4 Start-up and controls

- On power-up the relay is forced OFF (HIGH on the active-LOW module) and all indicators are off, so the system starts in a safe state.
- START** arms the system; **RESET** clears the latch and the manual fault; **TEST** forces a fault to prove the protection path; **MENU** is reserved.
- If the fault condition is still present after RESET, it latches again on the next 500 ms sample.

6.5 UART protocol to the FPGA

The ESP32 sends one ASCII line per loop iteration. Values are integers so the FPGA never needs floating-point arithmetic.

V=5012, I=640, T=2875, F=0, S=1\r\n

Field	Meaning	Unit / values
V	Voltage	millivolts (5.012 V = 5012)
I	Current	milliamps (0.640 A = 640)
T	Temperature	hundredths of a degree C (28.75 C = 2875)
F	ESP32 fault latched	1 = fault, 0 = no fault
S	Sensor status	1 = temperature sensor OK, 0 = sensor fault

7. FPGA Design (PYNQ-Z2, Verilog / Vivado)

The module *fpga_safety_led* takes the PYNQ-Z2 125 MHz clock and the UART receive line and produces four outputs: *led*, *relay_cutoff*, *warning_status* and *fault_status*.

7.1 UART receiver

- Two-flop synchroniser on the RX line to avoid metastability.
- Baud divider of 1085 clocks (125 MHz / 115200 = 1085.07), 8N1 framing. The line is sampled in the middle of each bit after verifying the start bit.
- States: IDLE, START, DATA, STOP. A byte is only accepted when the stop bit is high.

7.2 Text parser

Each received character updates a small parser. A letter (V, I, T, F or S) selects the current field, '=' starts a number, digits are accumulated as $\text{number} = \text{number} \times 10 + \text{digit}$, and a comma, CR or LF commits the value to the matching register. This lets the FPGA decode the human-readable frame directly, which also makes the link easy to debug with a serial monitor.

7.3 Thresholds and anomaly detection

Check	Condition	Result
Over-voltage	V above 5500 mV	FAULT
Over-current	I above 7500 mA	FAULT

Check	Condition	Result
Over-temperature	T above 40.00 deg C	FAULT
ESP32 fault flag	F not equal to 0	FAULT
Sensor failure	S equal to 0	FAULT
Voltage warning	V above 5000 mV	WARNING
Current warning	I above 6500 mA	WARNING
Temperature warning	T above 32.00 deg C	WARNING
Voltage spike	Change between consecutive samples above 500 mV	WARNING
Current surge	Change between consecutive samples above 1000 mA	WARNING
Rapid temperature rise (dT/dt)	Rise of more than 1.00 deg C between samples	WARNING

The anomaly checks are what separate the FPGA layer from a simple limit comparator: a value can still be within limits while its *rate of change* already signals trouble, such as a heating battery or a failing adapter. The FPGA also computes instantaneous power (mV x mA, in microwatts) for future use.

7.4 Safety FSM

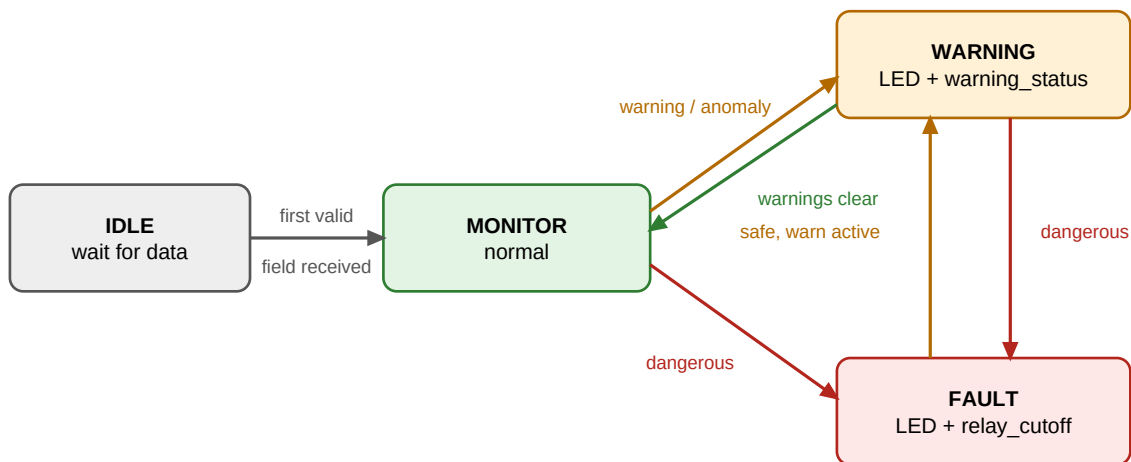


Figure 2. FPGA safety state machine

State	Meaning	led	warning_status	fault_status	relay_cutoff
IDLE	Waiting for the first valid data	0	0	0	0
MONITOR	All values normal	0	0	0	0
WARNING	Approaching a limit or anomaly seen	1	1	0	0
FAULT	Dangerous condition	1	0	1	1

In this testing version the FPGA leaves FAULT automatically once all dangerous conditions clear. The permanent latch lives in the ESP32, so after a fault the FPGA keeps reporting FAULT until the ESP32 fault flag (F) is cleared by the RESET button.

8. Two-Layer Protection Concept

The two controllers are deliberately different technologies so a single bug or failure is less likely to defeat both.

Aspect	ESP32 (software layer)	FPGA (hardware layer)
Implementation	Arduino C++ firmware, sequential loop	Verilog FSM, parallel logic
Inputs	Direct sensor measurements and user inputs	UART frame from the ESP32
Fault behaviour	Latches until RESET; drives relay directly	Follows conditions; drives relay_cutoff output
Extra detection	Sensor disconnect, manual test, pot injection	Warning limits, spikes, surges, dT/dt, ESP32 flag
User interface	LCD, LEDs, buzzer, buttons	LED and status output pins

9. Testing Methodology and Test Plan

All testing is done with a dummy battery. The three potentiometers let any protection path be triggered without heating or over-driving real hardware, and the TEST button forces a fault on demand. The table lists each test case with the behaviour the design is expected to show. The last column is for the measured observations from the bench.

#	Test case	How applied	Expected behaviour	Observed
1	Normal operation	Press START, values within limits	Green LED, relay ON, FPGA in MONITOR, no status outputs	
2	Temperature warning	Heat sensor to 32 deg C or more	Yellow LED, relay stays ON, FPGA WARNING with LED on	
3	Over-temperature	Temperature above 40 deg C	ESP32 latches fault, relay OFF, red LED and buzzer; FPGA FAULT, relay_cutoff = 1	
4	Over-voltage	Voltage above 5.5 V	Same fault response as test 3	
5	Over-current	Raise current / use current fault pot	Same fault response as test 3	
6	Pot fault injection	Any fault pot to 90 % or more	ESP32 fault latch, F=1 sent, FPGA FAULT	
7	Pot warning	Any pot to 70 % or more	Yellow LED, relay ON	
8	Manual test	Press TEST	Fault latched, relay OFF	
9	Sensor disconnect	Unplug DS18B20	T = -127 deg C, S = 0, both layers fault, LCD shows TEMP SENSOR ERR	
10	Voltage spike / current surge	Step change above 500 mV / 1000 mA	FPGA enters WARNING	
11	Rapid temperature rise	Rise above 1 deg C between samples	FPGA enters WARNING	
12	Recovery	Clear condition, press RESET	ESP32 latch cleared, relay can be re-enabled, FPGA returns to MONITOR	

10. Limitations and Future Scope

10.1 Known limitations of the prototype

- **Current range:** the fault limit of 7.5 A is above the 5 A range of the ACS712-5A module, so the sensor saturates before the limit can be reached. The limit should be set below 5 A, or a higher-range sensor used.
- **No data watchdog:** if UART data stops, the FPGA keeps using the last received values and does not fault. A timeout counter that forces FAULT on data loss is needed.
- **No hysteresis or latch in the FPGA:** the FPGA clears FAULT as soon as readings recover, so a value hovering near a limit can toggle the output. The ESP32 latch covers this in the current design.

- **Warning exit condition:** spike, surge and rapid-rise flags raise WARNING, but the WARNING exit test only checks the static warning limits, so the FSM can bounce between MONITOR and WARNING while an anomaly flag is set.
- **Threshold mismatch:** the ESP32 warns on temperature only (plus pots), while the FPGA also warns on voltage and current; the two layers do not share exactly the same warning logic.
- **Negative temperature values:** the FPGA parser ignores the minus sign, so -127 deg C is read as 127 deg C. It still faults, but it is detected through the S flag and a very high reading, not through its true value.
- **Continuous UART transmission:** frames are sent every loop pass, even though measurements only change every 500 ms.
- **Prototype scale:** tested on a dummy battery with potentiometer-simulated faults, not on real lithium cells.

10.2 Future work

- Add a data-valid watchdog, FPGA fault latch with hysteresis, and a checksum on the UART frame.
- Use a higher-range current sensor and calibrate against a precision meter at several load points.
- Add a second temperature sensor and a thermal-runaway model that uses dT/dt and charge level together.
- Integrate the logic into a compact PCB or custom ASIC / low-cost MCU version for real adapters.
- Add wireless alerts (the ESP32 already supports Wi-Fi and Bluetooth) and logging of fault events.
- Explore a learned anomaly detector trained on charging profiles.

11. Conclusion

The Safe Charging Chip demonstrates that a low-cost, dual-layer safeguard can watch voltage, current and temperature together, warn the user early and physically disconnect the battery on a fault. The ESP32 provides robust acquisition, a latching fault memory and user interface, while the FPGA adds an independent, hardware-timed second opinion that also watches how quickly values are changing. With the identified limitations addressed, the design is a practical starting point for safer mobile charging.

Appendix A. Parameter Summary

Parameter	Value
FPGA clock / UART	125 MHz / 115200 baud, 8N1, divider 1085
Fault limits	5.5 V, 7.5 A, 40.0 deg C
Warning limits (FPGA)	5.0 V, 6.5 A, 32.0 deg C
Spike / surge / dT limits	500 mV, 1000 mA, 1.00 deg C per sample
ACS712	Zero 1749.5 mV, 185 mV/A, divider factor 1.5
Voltage module ratio	5:1
ESP32 sampling / LCD refresh	500 ms / 2 s
Pot thresholds	Warning at 70 %, fault at 90 %
Relay logic	ESP32: active LOW (LOW = ON)